

**ПРАВИТЕЛЬСТВО РОССИЙСКОЙ ФЕДЕРАЦИИ
ФЕДЕРАЛЬНОЕ ГОСУДАРСТВЕННОЕ АВТОНОМНОЕ
ОБРАЗОВАТЕЛЬНОЕ УЧРЕЖДЕНИЕ ВЫСШЕГО ОБРАЗОВАНИЯ
НАЦИОНАЛЬНЫЙ ИССЛЕДОВАТЕЛЬСКИЙ УНИВЕРСИТЕТ
«ВЫСШАЯ ШКОЛА ЭКОНОМИКИ»**

Факультет компьютерных наук

Образовательная программа «Дизайн и разработка информационных продуктов»

СОГЛАСОВАНО

Научный руководитель
Приглашённый преподаватель:
Факультет компьютерных наук /
Департамент больших данных и
информационного поиска / Базовая
кафедра Т-Банка

_____ А. А. Шкель
«__» _____ 2026 г.

УТВЕРЖДЕНО

Академический руководитель
образовательной программы
«Дизайн и разработка
информационных продуктов»,
преподаватель базовой кафедры Т-
Банка

_____ Д. Д. Син
«__» _____ 2026 г.

ПЛАТФОРМА ДЛЯ NO-CODE АВТОМАТИЗАЦИИ БИЗНЕС-ПРОЦЕССОВ.

Программа и методика испытаний (индивидуальная часть)

ЛИСТ УТВЕРЖДЕНИЯ

RU.17701729.09.03-04 51 03-1-ЛУ

Исполнители:

Студент группы БДРИП242

_____ / А. А. Клушин /

«__» _____ 2026 г.

Москва 2026

Подп. и дата	
Инв.№ дубл.	
Взам. инв.№	
Подп. и дата	
Инв.№ подл.	

УТВЕРЖДЕН

RU.17701729.09.03-04 51 03-1-ЛУ

ПЛАТФОРМА ДЛЯ NO-CODE АВТОМАТИЗАЦИИ БИЗНЕС-ПРОЦЕССОВ.

Программа и методика испытаний (индивидуальная часть)

RU.17701729.09.03-04 51 03-1

Листов 24

Инов.№ подп	Подп. и дата	Взам. инв.№	Инов.№ дубл.	Подп. и дата

АННОТАЦИЯ

Настоящий документ является индивидуальной частью программы и методики испытаний программы «Платформа для no-code автоматизации бизнес-процессов» в рамках командной курсовой работы (общая часть — RU.17701729.09.03-04 51 01-1). Документ определяет методики испытаний серверной части, исполнителей узлов, песочницы Code Node, отказоустойчивости и автоматической документации REST API, выполняемых студентом группы БДРИП242 Клушиным А.А.

Настоящий документ разработан в соответствии с ГОСТ 19.301-79.

Изм.	Лист	№ докум.	Подп.	Дата
RU.17701729.09.03-04 51 03-1				
Инв. № подл.	Подп. и дата	Взам. Инв. №	Инв. № дубл.	Подп. и дата

СОДЕРЖАНИЕ

1. ОБЪЕКТ ИСПЫТАНИЙ	5
1.1. Наименование программы	5
1.2. Краткая характеристика	5
2. ЦЕЛЬ ИСПЫТАНИЙ	6
3. СРЕДСТВА И ПОРЯДОК ИСПЫТАНИЙ	7
3.1. Технические средства	7
3.2. Программные средства	7
3.3. Порядок проведения испытаний	7
4. СОСТАВ АВТОМАТИЧЕСКИХ ТЕСТОВ СЕРВЕРНОЙ ЧАСТИ	9
4.1. Перечень тестовых классов	9
4.2. Запуск автоматических тестов	10
4.3. Описание интеграционных тестов	10
5. МЕТОДЫ ИСПЫТАНИЙ ФУНКЦИОНАЛЬНЫХ ТРЕБОВАНИЙ	11
5.1. Испытание системы триггеров (требование ТЗ-2.1.1)	11
5.2. Испытание HTTP-узла (требование ТЗ-2.1.2)	12
5.3. Испытание узлов обработки потоков данных (требование ТЗ-2.1.3)	12
5.4. Испытание изолированного исполнения Code Node (требование ТЗ-2.1.4)	13
5.5. Испытание параллельного исполнения и отказоустойчивости (требование ТЗ-2.1.5) ...	14
5.6. Испытание версионирования и снапшотов (требование ТЗ-2.1.6 и ТЗ-2.1.13)	15
5.7. Испытание трансляции событий по WebSocket (требование ТЗ-2.1.10, серверная сторона)	16
5.8. Испытание контент-адресуемого хранения в MinIO (требование ТЗ-2.1.11)	17
5.9. Испытание серверного модуля А/В-аналитики (требование ТЗ-2.1.12)	17
5.10. Испытание режима пошаговой отладки workflow (требование ТЗ-2.1.14)	18
6. ИСПЫТАНИЕ НЕФУНКЦИОНАЛЬНЫХ ТРЕБОВАНИЙ	19
6.1. Испытание производительности REST API	19
6.2. Испытание автоматической документации REST API	19
6.3. Испытание целевого покрытия по JaCoCo	20
7. СВОДНЫЕ ПОКАЗАТЕЛИ	21

Изм.	Лист	№ докум.	Подп.	Дата
RU.17701729.09.03-04 51 03-1				
Инв. № подл.	Подп. и дата	Взам. Инв. №	Инв. № дубл.	Подп. и дата

8. ПРИЛОЖЕНИЕ. ТРАССИРОВКА ИНДИВИДУАЛЬНЫХ ТРЕБОВАНИЙ К РАЗДЕЛАМ	
ПМИ	22
СПИСОК ИСПОЛЬЗОВАННЫХ ИСТОЧНИКОВ	23

Изм.	Лист	№ докум.	Подп.	Дата
RU.17701729.09.03-04 51 03-1				
Инв. № подл.	Подп. и дата	Взам. Инв. №	Инв. № дубл.	Подп. и дата

1. ОБЪЕКТ ИСПЫТАНИЙ

1.1. Наименование программы

Объектом испытаний является серверная часть программного продукта «Платформа для no-code автоматизации бизнес-процессов» (условное обозначение — «FluxPilot»), разрабатываемая в индивидуальной части курсовой работы студентом группы БДРИП242 Клушиным А.А.

1.2. Краткая характеристика

Серверная часть реализована на языке программирования Kotlin версии 2.2.21 с использованием каркаса Spring Boot версии 3.5.3, исполняемого на платформе JDK 24. Серверная часть содержит ядро исполнения workflow WorkflowExecutionService с топологической сортировкой графа по алгоритму Кана и параллельным исполнением через интерфейс CompletableFuture, исполнители узлов десяти типов (HttpNodeExecutor, пять реализаций DataflowNodeExecutors для операций обработки потоков данных, PythonNodeExecutor и JavaScriptNodeExecutor для исполнения пользовательского кода, BranchSplitNodeExecutor и BranchMergeNodeExecutor для узлов разделения и слияния ветвей A/B-эксперимента, внутренний TriggerWebhookExecutor), общий движок Docker-песочницы ContainerSandboxRunner с предзагрузчиком образов SandboxImageWarmer, сервис триггеров с планировщиком cron-задач и обработчиком webhook-запросов, сервис версионирования с двухуровневой схемой ревизий и именованных версий, сервис именованных снапшотов уровня документа, сервис контент-адресуемого хранения BlobService с утилитами канонизации JSON и вычисления SHA-256, серверный модуль продуктовой A/B-аналитики со статистическим тестом разности конверсий, WebSocket-брокер с трансляцией событий жизненного цикла исполнения, миграции схемы реляционной базы данных PostgreSQL средствами Liquibase, автоматическую документацию REST API средствами springdoc-openapi.

Изм.	Лист	№ докум.	Подп.	Дата
RU.17701729.09.03-04 51 03-1				
Инв. № подл.	Подп. и дата	Взам. Инв. №	Инв. № дубл.	Подп. и дата

2. ЦЕЛЬ ИСПЫТАНИЙ

Целью испытаний является подтверждение соответствия серверной части требованиям, изложенным в индивидуальной части технического задания (RU.17701729.09.03-04 ТЗ 03-1), а также проверка достижения целевых нефункциональных показателей, заявленных в индивидуальной части пояснительной записки (RU.17701729.09.03-04 81 03-1, раздел «Ожидаемые качественные и количественные показатели»). К проверке подлежат корректность параллельного исполнения и обработки сбоев независимых ветвей workflow, полнота изоляции пользовательского Python- и JavaScript-кода в Docker-песочнице, идемпотентность и эффективность дедупликации контент-адресуемого хранения в MinIO, атомарность и иммутабельность операций версионирования и отката, корректность работы трёх типов триггеров запуска workflow, корректность серверного модуля А/В-аналитики и статистического тестирования, целевые показатели производительности и автоматизированного тестирования.

Изм.	Лист	№ докум.	Подп.	Дата
RU.17701729.09.03-04 51 03-1				
Инв. № подл.	Подп. и дата	Взам. Инв. №	Инв. № дубл.	Подп. и дата

3. СРЕДСТВА И ПОРЯДОК ИСПЫТАНИЙ

3.1. Технические средства

Для проведения испытаний требуется один компьютер с процессором архитектуры x86_64 не менее 4 ядер, оперативной памятью не менее 8 ГБ (рекомендуется 16 ГБ для одновременного запуска серверной части, клиентской части, реляционной базы данных PostgreSQL, объектного хранилища MinIO, Docker-контейнеров изолированной песочницы и средств мониторинга), дисковым пространством не менее 20 ГБ, стабильным подключением к сети Интернет для первоначальной загрузки Docker-образов `python:3.11-slim` и `node:20-alpine`, используемых песочницей Code Node. Операционная система — Linux (Ubuntu 22.04 LTS и совместимые дистрибутивы), macOS версии 12 и выше или Windows версии 10/11 с подсистемой WSL2.

3.2. Программные средства

Для проведения испытаний требуются следующие программные средства: JDK 24 для запуска fat-JAR серверной части; Apache Maven версии 3.9 или выше для пересборки из исходных кодов; Docker Engine версии 20.10 или выше с активным `docker daemon`, необходимый для интеграционных тестов через библиотеку `Testcontainers` и для исполнения узлов Code Node; PostgreSQL версии 16, поднимаемый через Docker Compose; MinIO стабильной версии, поднимаемый через Docker Compose; утилита `curl` или альтернативные средства (HTTPIe, Postman) для выполнения HTTP-запросов к REST API; современный веб-браузер (Google Chrome версии 90 и выше, Mozilla Firefox версии 90 и выше) для работы с интерфейсом Swagger UI и веб-консолью MinIO; утилита `psql` для выполнения прямых SQL-запросов к базе данных при проверках состояния.

3.3. Порядок проведения испытаний

Испытания проводятся в три этапа. Первый этап заключается в запуске полного цикла автоматических модульных и интеграционных тестов серверной части командой `cd backend && mvn verify`. Время прогона на референсном железе не превышает 8 минут. По завершении выполняется анализ отчётов `backend/target/surefire-reports/` (JUnit XML) и `backend/target/site/jacoco/index.html` (HTML-отчёт покрытия). Критерием успешного прохождения первого этапа является 100-процентное завершение всех тестов со статусом `success` и покрытие по строкам не ниже целевого порога (90%).

Изм.	Лист	№ докум.	Подп.	Дата
RU.17701729.09.03-04 51 03-1				
Инв. № подл.	Подп. и дата	Взам. Инв. №	Инв. № дубл.	Подп. и дата

Второй этап заключается в ручной проверке функциональных характеристик по методикам раздела 5 настоящего документа. Для каждой методики последовательно выполняются перечисленные действия, фиксируются фактические результаты, выполняется сравнение с критериями успеха.

Третий этап заключается в проверке нефункциональных характеристик в соответствии с разделом 6 настоящего документа: измерение характеристик производительности через curl и Swagger UI, проверка безопасности и изоляции Docker-песочницы серией атакующих сценариев, проверка автоматической документации REST API.

Изм.	Лист	№ докум.	Подп.	Дата
RU.17701729.09.03-04 51 03-1				
Инв. № подл.	Подп. и дата	Взам. Инв. №	Инв. № дубл.	Подп. и дата

4. СОСТАВ АВТОМАТИЧЕСКИХ ТЕСТОВ СЕРВЕРНОЙ ЧАСТИ

4.1. Перечень тестовых классов

Автоматические тесты серверной части размещены в директории backend/src/test/kotlin/. Состав тестовых классов перечислен в таблице 1; сводные количественные показатели приведены в разделе «Сводные показатели» настоящего документа.

Класс тестов	Тип
DataflowNodeExecutorsTest	Модульные тесты пяти операций обработки потоков данных (JUnit Jupiter)
MvpIntegrationTests	Интеграционные тесты сквозных сценариев (Testcontainers с реальными PostgreSQL и MinIO)
AllaApplicationTests	Тесты загрузки контекста Spring и базовых health-эндпойнтов
BranchSplitNodeExecutorTest	Модульные тесты узла разделения потока на ветви
BranchSplitStrategiesTest	Модульные тесты стратегий разделения (random, hash, modulo, attribute, percentage, stratified)
SplitEnvelopeTest	Модульные тесты обёртки SplitEnvelope над выходом BranchSplit
BranchMergeNodeExecutorTest	Модульные тесты узла слияния ветвей
WorkflowExecutionServiceBranchTest	Интеграционные тесты исполнения с BranchSplit и BranchMerge
AbAnalyticsServiceTest	Модульные тесты сервиса агрегации А/В-аналитики
AbAnalyticsControllerTest	Интеграционные тесты REST-эндпойнта /v1/workflows/{id}/ab-analytics
StatTestTest	Модульные тесты статистической проверки разности конверсий
ConnectionSkeletonTest	Модульные тесты модели соединений графа workflow

Таблица 1. Состав автоматических тестов серверной части

В состав модульных тестов дополнительно входят тесты исполнителя HttpNodeExecutor с применением MockWebServer, тесты сервиса контент-адресуемого хранения BlobService через библиотеку localstack или контейнер MinIO в Testcontainers, тесты планировщика TriggerScheduler с подменой системного времени на Clock.fixed, тесты сервиса исполнения WorkflowExecutionService с мок-исполнителями узлов для проверки логики обхода направлен-

Изм.	Лист	№ докум.	Подп.	Дата
RU.17701729.09.03-04 51 03-1				
Инв. № подл.	Подп. и дата	Взам. Инв. №	Инв. № дубл.	Подп. и дата

ного ациклического графа и обработки сбоев, тесты утилит CanonicalJson и Hashing (чистые функции с проверкой edge-cases), тесты доменного сервиса WorkflowService с проверкой иммутабельности ревизий и корректности отката.

4.2. Запуск автоматических тестов

Запуск всех тестов с генерацией отчёта покрытия выполняется командой `cd backend && mvn verify`. Запуск только модульных тестов выполняется командой `mvn test`. Запуск одного тестового класса выполняется командой `mvn test -Dtest=DataflowNodeExecutorsTest`. Запуск одного тестового метода выполняется командой `mvn test -Dtest=DataflowNodeExecutorsTest#filterRetainsMatchingItems`. Просмотр HTML-отчёта покрытия после выполнения команды `mvn verify` выполняется открытием файла `backend/target/site/jacoco/index.html` в браузере.

4.3. Описание интеграционных тестов

Тестовый класс `MvpIntegrationTests` использует библиотеку `Testcontainers` для автоматического запуска реальных контейнеров PostgreSQL 16 и MinIO на время прогона тестов. Контейнеры объявляются как поля с аннотацией `@Container` и модификатором `static`, что обеспечивает их переиспользование между тестами одного класса с изоляцией состояния через очистку таблиц в методе с аннотацией `@BeforeEach`. Тестовый Spring-контекст конфигурируется через `@SpringBootTest(webEnvironment = RANDOM_PORT)` с переопределением свойств `spring.datasource.url` и `app.s3.endpoint` через `@DynamicPropertySource` на адреса контейнеров. В состав тестов класса входят `testCreateWorkflowFlow` (создание workflow с проверкой появления записи в таблице `workflows` и корректности возврата DTO), `testRunSimpleWorkflowSuccess` (создание workflow с одним HTTP-узлом, запуск, проверка статуса `success` и сохранения `output`), `testParallelBranchesWithFailure` (создание workflow с двумя параллельными ветвями, одна из которых заведомо сбойная, запуск, проверка изоляции сбоя и продолжения параллельной ветви), `testBlobDeduplicationAcrossRevisions` (создание двух ревизий workflow с идентичным узлом, проверка единственности записи в таблице `blob_index` и единственности объекта в MinIO), `testRestoreVersionRecreatesRevision` (создание трёх ревизий, создание именованной версии на первой ревизии, восстановление, проверка создания новой ревизии с прошлым содержимым графа), `testWebhookTriggerCreatesRun` (создание webhook-триггера, отправка POST-запроса на сгенерированный URL, проверка создания записи запуска и корректной передачи входного тела).

Изм.	Лист	№ докум.	Подп.	Дата
RU.17701729.09.03-04 51 03-1				
Инв. № подл.	Подп. и дата	Взам. Инв. №	Инв. № дубл.	Подп. и дата

5. МЕТОДЫ ИСПЫТАНИЙ ФУНКЦИОНАЛЬНЫХ ТРЕБОВАНИЙ

5.1. Испытание системы триггеров (требование ТЗ-2.1.1)

Целью данной методики является подтверждение корректности работы трёх типов триггеров запуска workflow: ручного запуска, scheduler-триггера на основе cron-выражения и webhook-триггера. Подготовка к проведению методики включает запуск инфраструктуры командой `docker compose -f deploy/docker-compose.dev.yml up -d`, запуск серверной части командой `cd backend && mvn spring-boot:run`, запуск клиентской части командой `cd frontend && npm start`, открытие редактора по адресу `http://localhost:4200` и создание нового workflow с одним HTTP-узлом, направленным на адрес `https://httpbin.org/get`, с последующим сохранением.

Для проверки ручного триггера в интерфейсе редактора нажимается кнопка «Запустить» в правом верхнем углу. Ожидаемый результат заключается в появлении жёлтой подсветки узла (статус `running`), переходе подсветки в зелёный цвет (статус `success`) в течение 5 секунд, появлении новой записи в панели «История запусков» со статусом `success`, отображением длительности исполнения и времени запуска. Альтернативный способ запуска через `curl` выполняется командой `curl -X POST http://localhost:8080/v1/workflows/{id}/runs`, которая возвращает ответ HTTP 202 Accepted с идентификатором созданного запуска; команда `curl http://localhost:8080/v1/workflow-runs/{runId}` возвращает текущее состояние запуска.

Для проверки scheduler-триггера в интерфейсе редактора открывается панель «Триггеры», нажимается кнопка «Добавить триггер», выбирается тип «Scheduler», в поле cron-выражения вводится значение `*/1 * * * *` (запуск каждую минуту), активируется флаг `enabled` и сохраняется конфигурация. После двух-трёх минут ожидания в панели «История запусков» появляются как минимум две записи запусков с интервалом в одну минуту, типом триггера `scheduler` и статусом `success`.

Для проверки webhook-триггера в интерфейсе редактора открывается панель «Триггеры», нажимается кнопка «Добавить триггер», выбирается тип «Webhook» и сохраняется конфигурация. Из ответа копируется сгенерированный URL формата `/v1/webhook/{token}`. Отправляется POST-запрос с произвольным JSON-телом командой `curl -X POST http://localhost:8080/v1/webhook/{token} -H "Content-Type: application/json" -d '{"hello":"world"}'`. Ожидаемый результат заключается в получении ответа HTTP 202 Accepted, появлении новой записи в панели «История запусков» со статусом `success` и типом триггера `webhook`, отображении переданного тела `{"hello":"world"}` в качестве входа первого узла workflow в детализации запуска.

Изм.	Лист	№ докум.	Подп.	Дата
RU.17701729.09.03-04 51 03-1				
Инв. № подл.	Подп. и дата	Взам. Инв. №	Инв. № дубл.	Подп. и дата

Критерием успешного прохождения методики является корректное инициирование запуска workflow всеми тремя типами триггеров и корректная фиксация статусов и входных данных в системе.

5.2. Испытание HTTP-узла (требование ТЗ-2.1.2)

Целью данной методики является подтверждение корректности работы исполнителя HTTP-узла для всех поддерживаемых HTTP-методов (GET, POST, PUT, PATCH, DELETE), корректной обработки настраиваемых заголовков, тела запроса и тайм-аута, а также диагностики ошибочных ситуаций.

В рамках методики последовательно создаются и запускаются следующие конфигурации workflow с одним HTTP-узлом. Первая конфигурация: метод GET, URL `https://httpbin.org/get?param=value`, без тела запроса. Ожидается статус success и наличие эхо-параметра в выходных данных. Вторая конфигурация: метод POST, URL `https://httpbin.org/post`, заголовки `Content-Type: application/json` и `Authorization: Bearer test-token`, тело запроса `{"name": "Артём", "group": "БДРИП242"}`. Ожидается статус success и эхо-заголовков и тела в выходных данных. Третья конфигурация: метод GET, URL `https://invalid-domain-does-not-exist-abcde12345.com`. Ожидается статус failed с диагностическим сообщением об ошибке DNS или невозможности установить соединение. Четвёртая конфигурация: метод GET, URL `https://httpstat.us/200?sleep=10000` (намеренная задержка 10 секунд), тайм-аут 3 секунды. Ожидается статус failed с диагностическим сообщением о превышении тайм-аута. Пятая конфигурация: метод DELETE, URL `https://httpbin.org/delete`. Ожидается статус success с эхо-методом DELETE в выходных данных.

Критерием успешного прохождения методики является корректное исполнение всех поддерживаемых HTTP-методов и корректная диагностика ошибочных ситуаций (ошибка DNS, превышение тайм-аута) с осмысленными сообщениями об ошибке.

5.3. Испытание узлов обработки потоков данных (требование ТЗ-2.1.3)

Целью данной методики является подтверждение корректности работы пяти операций обработки потоков данных. Покрытие обеспечивается модульным тестом `DataflowNodeExecutorsTest`. Запуск автоматического теста выполняется командой `cd backend && mvn test -Dtest=DataflowNodeExecutorsTest`. Ожидаемый результат заключается в успешном прохождении всех тестовых методов класса. Отчёт `target/`

Изм.	Лист	№ докум.	Подп.	Дата
RU.17701729.09.03-04 51 03-1				
Инв. № подл.	Подп. и дата	Взам. Инв. №	Инв. № дубл.	Подп. и дата

surefire-reports/ содержит подробности по каждому методу с именами, соответствующими проверяемым сценариям: `filterRetainsMatchingItems`, `mapAppliesTransformationToEach`, `reduceAccumulatesWithInitial`, `foreachExecutesSideEffectForEach`, `flatMapFlattensNestedLists` и других.

Дополнительная ручная проверка через интерфейс редактора выполняется созданием workflow с конфигурацией HTTP-узла, направленный на API, возвращающее список объектов, → Filter-узел с выражением `item.age > 18`. После запуска workflow ожидается, что в выходных данных Filter-узла остались только объекты с полем `age`, превышающим значение 18.

Критерием успешного прохождения методики является 100-процентное успешное выполнение всех автоматических тестов класса `DataflowNodeExecutorsTest` и подтверждение корректной работы хотя бы одной операции в составе полноценного workflow через интерфейс редактора.

5.4. Испытание изолированного исполнения Code Node (требование ТЗ-2.1.4)

Целью данной методики является подтверждение полноты изоляции пользовательского кода в Docker-песочнице. Проверки разделяются на функциональные (корректный код выполняется без сбоев) и безопасностные (атакующие сценарии блокируются с осмысленными диагностическими сообщениями).

В рамках функциональной проверки создаётся workflow с одним узлом Code Node на языке Python с кодом, читающим JSON из стандартного потока ввода, вычисляющим сумму элементов входного массива `numbers` и печатающим результат в стандартный поток вывода. Запуск с входными данными `{"numbers": [1,2,3,4,5]}` ожидаемо завершается со статусом `success` и выходными данными `{"sum": 15}`. Аналогичная проверка выполняется для узла Code Node на языке JavaScript с эквивалентным кодом.

В рамках безопасностной проверки последовательно создаются и запускаются workflow с узлами Code Node, реализующими следующие атакующие сценарии. Попытка сетевого обращения через `urllib.request.urlopen` для Python и эквивалентный код для JavaScript должна завершаться со статусом `failed` и диагностическим сообщением об отсутствии сетевого интерфейса (что подтверждает действие флага `--network none`). Попытка записи в файловую систему через `open("/etc/test", "w")` должна завершаться со статусом `failed` и диагностическим сообщением о режиме только для чтения файловой системы (что подтверждает действие флага `--read-only`). Попытка выделения значительного объёма памяти через выражение `x = [0] * (10**8)` должна завершаться со статусом `failed` и диагностическим сообщением об исчерпании

Изм.	Лист	№ докум.	Подп.	Дата
RU.17701729.09.03-04 51 03-1				
Инв. № подл.	Подп. и дата	Взам. Инв. №	Инв. № дубл.	Подп. и дата

памяти OOM (что подтверждает действие флага `--memory 256m`). Попытка `fork-bomb` через бесконечный вызов `os.fork()` должна завершаться со статусом `failed` и диагностическим сообщением о превышении лимита процессов (что подтверждает действие флага `--pids-limit 64`). Попытка бесконечного цикла должна завершаться со статусом `failed` и диагностическим сообщением о превышении тайм-аута через 5 секунд (что подтверждает действие тайм-аута). Попытка повышения привилегий через `subprocess.run(["sudo", "id"])` должна завершаться со статусом `failed` (что подтверждает действие флагов `--cap-drop ALL` и `--security-opt no-new-privileges`).

Критерием успешного прохождения методики является корректное исполнение функциональной проверки и блокировка всех шести атакующих сценариев с диагностическими сообщениями.

5.5. Испытание параллельного исполнения и отказоустойчивости (требование ТЗ-2.1.5)

Целью данной методики является подтверждение параллельного исполнения независимых ветвей `workflow` и изоляции сбоев одной ветви от параллельных.

В рамках проверки параллелизма создаётся `workflow` со структурой: триггер связан с двумя независимыми узлами `Code Node` — узлом А на Python с задержкой 3 секунды и узлом В на JavaScript с задержкой 3 секунды; узлы А и В каждый связан с собственным узлом-потомком. После запуска ожидается одновременное появление жёлтой подсветки узлов А и В, переход обоих узлов в зелёное состояние почти одновременно через приблизительно 3 секунды, после чего активируются узлы-потомки. Общее время исполнения `workflow` приближённо составляет 6 секунд, а не 12 секунд, что подтверждается анализом значений времени старта в записях узлов запуска через SQL-запрос или REST API.

В рамках проверки изоляции сбоев в параллельных ветвях создаётся `workflow` со структурой: триггер связан с двумя независимыми узлами — узлом А в виде HTTP-узла с заведомо невалидным URL и узлом В в виде корректного узла `Code Node`; каждый узел связан с собственным узлом-потомком. После запуска ожидается переход узла А в красное состояние (статус `failed`), переход узла-потомка узла А в серое состояние (статус `skipped` с диагностическим сообщением о неуспешной зависимости), переход узла В и его потомка в зелёное состояние (статус `success`), агрегированный статус `workflow` — `failed`.

В рамках проверки каскадного пропуска по длинной цепочке создаётся `workflow` со структурой $A \rightarrow B \rightarrow C \rightarrow D$, в котором узел В заведомо сбоит. После запуска ожидается переход узла

Изм.	Лист	№ докум.	Подп.	Дата
RU.17701729.09.03-04 51 03-1				
Инв. № подл.	Подп. и дата	Взам. Инв. №	Инв. № дубл.	Подп. и дата

А в зелёное состояние, узла В в красное состояние, узлов С и D в серое состояние с транзитивным распространением статуса skipped.

В рамках проверки детекции циклов через прямой SQL-апдейт или специальный тестовый REST-эндпоинт формируется workflow с циклом ($A \rightarrow B \rightarrow A$) и запускается командой `curl -X POST http://localhost:8080/v1/workflows/{id}/runs`. Ожидается статус failed с диагностическим сообщением «Cycle detected in workflow graph», отсутствие создания записей узлов запуска в статусе running.

Критерием успешного прохождения методики является корректное поведение во всех четырёх сценариях.

5.6. Испытание версионирования и снапшотов (требование ТЗ-2.1.6 и ТЗ-2.1.13)

Целью данной методики является подтверждение корректности автоматического создания ревизий при каждом сохранении, иммутабельности именованных версий и снапшотов, корректности операции отката.

В рамках методики последовательно выполняются следующие действия. Создаётся workflow с условным наименованием Test-versioning и сохраняется (создаётся ревизия 1). Через REST-запрос `curl http://localhost:8080/v1/workflows/{id}` фиксируется значение поля `current_version_id`. В интерфейсе редактора изменяется параметр одного узла, сохраняется (создаётся ревизия 2). Изменение и сохранение повторяются (создаётся ревизия 3). Создаётся именованная версия командой `curl -X POST http://localhost:8080/v1/workflows/{id}/versions -d '{"versionTag":"v1.0"}'`. Возвращается идентификатор созданной версии; команда `curl http://localhost:8080/v1/workflows/{id}/versions` возвращает список с одной версией с тегом v1.0. Конфигурация узла изменяется ещё раз и сохраняется (создаётся ревизия 4). Выполняется откат командой `curl -X POST http://localhost:8080/v1/workflows/{id}/versions/{vid}/restore`. Ожидается создание ревизии 5 с тем же содержимым графа, что у ревизии 1, на которой была создана версия v1.0; значение поля `current_version_id` workflow обновляется на ссылку на новую draft-версию, ссылающуюся на ревизию 5.

Для проверки иммутабельности через прямой SQL-запрос `SELECT revision_number, graph_skeleton FROM workflow_revision WHERE workflow_id = '<id>' ORDER BY revision_number`; ожидается наличие пяти записей; пятая идентична первой; вторая, третья и четвёртая остались без изменений с момента их создания. Для проверки защиты целостности выполняется попытка удаления ревизии, на которую ссылается именованная версия, командой

Изм.	Лист	№ докум.	Подп.	Дата
RU.17701729.09.03-04 51 03-1				
Инв. № подл.	Подп. и дата	Взам. Инв. №	Инв. № дубл.	Подп. и дата

DELETE FROM workflow_revision WHERE id = <revision1_id>; ожидается ошибка нарушения ограничения внешнего ключа (защита ON DELETE RESTRICT).

Дополнительно проверяется механизм именованных снапшотов уровня документа. Создаётся снапшот командой `curl -X POST http://localhost:8080/v1/workflows/{id}/snapshots -d '{"name":"milestone-1","description":"first stable release"}'`. Ожидается успешный ответ с идентификатором созданного снапшота. Команда `curl http://localhost:8080/v1/workflows/{id}/snapshots` возвращает список снапшотов с указанием имени и описания.

Критерием успешного прохождения методики является последовательное создание ревизий, сохранение иммутабельности исторических ревизий, корректное выполнение операции отката с созданием новой ревизии и успешное создание именованных снапшотов уровня документа.

5.7. Испытание трансляции событий по WebSocket (требование ТЗ-2.1.10, серверная сторона)

Целью данной методики является подтверждение публикации серверной частью событий жизненного цикла исполнения в STOMP-топик `/topic/workflows/{workflowId}/graph` через WebSocket-канал.

В рамках методики выполняется подключение к WebSocket-каналу через STOMP-клиент, например через утилиту `stompit-cli`, с подпиской на топик командой `SUBSCRIBE id=sub-0 destination=/topic/workflows/{workflowId}/graph`. В отдельном терминале запускается workflow командой `curl -X POST http://localhost:8080/v1/workflows/{id}/runs`. Ожидается последовательное поступление на подписанный клиент сообщений с типами `workflow_started`, `node_reached` (для каждого узла), `node_exited` (для каждого узла со статусом исполнения), `workflow_finished`. Каждое сообщение содержит корректные идентификаторы узла, статус и временные метки. Альтернативная проверка выполняется в браузере через инструменты разработчика: открывается раздел Network, фильтр на WS, наблюдаются STOMP-кадры при запуске workflow.

Критерием успешного прохождения методики является публикация событий в правильном порядке без потерь с корректным заполнением полей.

Изм.	Лист	№ докум.	Подп.	Дата
RU.17701729.09.03-04 51 03-1				
Инв. № подл.	Подп. и дата	Взам. Инв. №	Инв. № дубл.	Подп. и дата

5.8. Испытание контент-адресуемого хранения в MinIO (требование ТЗ-2.1.11)

Целью данной методики является подтверждение дедупликации идентичных конфигураций узлов между ревизиями и workflow.

В рамках методики создаётся workflow A с одним HTTP-узлом, имеющим конфигурацию URL `https://example.com`, и сохраняется (создаётся ревизия 1). Затем создаётся workflow B с идентичным HTTP-узлом и сохраняется. Через SQL-запрос `SELECT content_hash, COUNT(*) FROM blob_index WHERE content_hash IN (<hash1>, <hash2>) GROUP BY content_hash`; ожидается получение единственной записи с `count = 1` (один уникальный хеш, не размноженный). Через веб-консоль MinIO по адресу `http://localhost:6006` в bucket `alla-blobs` в директории `blobs/` ожидается отсутствие дублирующих файлов с одинаковым размером.

Изменяется конфигурация узла в workflow A (например, URL заменяется на `https://example.org`), сохраняется (создаётся ревизия 2). Через SQL-запрос ожидается появление одной новой записи в таблице `blob_index` с новым хешем для изменённой конфигурации; запись для исходного хеша остаётся неизменной, поскольку используется ревизией 1 workflow A и единственной ревизией workflow B.

Критерием успешного прохождения методики является однократное хранение идентичных конфигураций в MinIO, раздельное хранение различных конфигураций и корректная фиксация хешей в индексной таблице.

5.9. Испытание серверного модуля А/В-аналитики (требование ТЗ-2.1.12)

Целью данной методики является подтверждение корректности работы узлов разделения и слияния ветвей А/В-эксперимента, корректности серверного модуля агрегации просмотров и конверсий, корректности статистического теста разности конверсий.

В рамках методики последовательно выполняются модульные и интеграционные тесты, входящие в состав автоматических тестов серверной части. Запуск тестов узлов разделения и слияния выполняется командой `mvn test -Dtest="BranchSplit*Test,BranchMerge*Test,SplitEnvelopeTest"`. Ожидается успешное прохождение всех тестов классов `BranchSplitNodeExecutorTest`, `BranchSplitStrategiesTest`, `SplitEnvelopeTest`, `BranchMergeNodeExecutorTest`. Запуск интеграционных тестов исполнения с разделением и слиянием выполняется командой `mvn test -Dtest=WorkflowExecutionServiceBranchTest`. Запуск тестов модуля аналитики выполняется командой `mvn test -Dtest="AbAnalyticsServiceTest,AbAnalyticsControllerTest,StatTestTest"`.

Изм.	Лист	№ докум.	Подп.	Дата
RU.17701729.09.03-04 51 03-1				
Инв. № подл.	Подп. и дата	Взам. Инв. №	Инв. № дубл.	Подп. и дата

Дополнительная ручная проверка через REST API выполняется обращением к эндпойнту `curl http://localhost:8080/v1/workflows/{workflowId}/ab-analytics` для workflow, содержащего узлы BranchSplit и BranchMerge с накопленной историей запусков. Ожидается возврат JSON-ответа, содержащего просмотры и конверсии по каждому варианту эксперимента, выборочные конверсии, доверительные интервалы и р-значение разности конверсий.

Критерием успешного прохождения методики является 100-процентное успешное выполнение всех автоматических тестов модуля А/В-аналитики и корректный формат ответа REST-эндпойнта аналитики.

5.10. Испытание режима пошаговой отладки workflow (требование ТЗ-2.1.14)

Целью данной методики является проверка корректности работы серверного механизма запуска одной выбранной ноды workflow с произвольным входным значением без полного прогона графа.

Порядок проведения испытания: создаётся workflow, состоящий из нескольких связанных узлов; для произвольной ноды в составе графа инициируется отладочный запуск с указанием идентификатора целевой ноды и произвольного JSON-тела в качестве входа; проверяется, что серверная часть возвращает результат исполнения в синхронном виде; через прямой SQL-запрос к таблице workflow_run проверяется создание записи запуска с пометкой отладочного исполнения; через запрос к таблице node_run проверяется наличие записи о результате исполнения только выбранной ноды и отсутствие записей о предшествующих узлах графа; повторяется проверка с заведомо некорректным входом для подтверждения корректной обработки ошибок и фиксации диагностического сообщения в поле error_message записи node_run.

Критерием успешного прохождения методики является исполнение только выбранной ноды без вовлечения предшествующих узлов графа, синхронный возврат результата клиентской части, корректное создание записей в таблицах workflow_run и node_run с пометкой отладочного исполнения, корректная фиксация диагностических сообщений при ошибочных входах.

Изм.	Лист	№ докум.	Подп.	Дата
RU.17701729.09.03-04 51 03-1				
Инв. № подл.	Подп. и дата	Взам. Инв. №	Инв. № дубл.	Подп. и дата

6. ИСПЫТАНИЕ НЕФУНКЦИОНАЛЬНЫХ ТРЕБОВАНИЙ

6.1. Испытание производительности REST API

Целью данной методики является подтверждение целевого показателя «95-й перцентиль времени отклика REST API на запросы чтения не превышает 500 мс при типовой нагрузке».

Для каждого из пяти ключевых эндпойнтов на чтение выполняется серия из 50 последовательных запросов через утилиту curl с замером значения `time_total`, после чего рассчитываются минимальное, среднее и 95-й перцентильное значения. Команда замера для одного эндпойнта: `for i in {1..50}; do curl -s -w "%{time_total}\n" -o /dev/null http://localhost:8080/v1/workflows; done | sort -n`; 95-й перцентиль соответствует 48-му значению отсортированной последовательности. Аналогичные замеры выполняются для эндпойнтов `/v1/workflows/{id}`, `/v1/workflows/{id}/runs`, `/v1/workflow-runs/{runId}`, `/v1/swagger-ui.html`. Результаты фиксируются в форме таблицы с указанием эндпойнта, минимального, среднего и 95-го перцентильного значения и заключения о соответствии целевому показателю.

Критерием успешного прохождения методики является непревышение значения 500 мс для 95-го перцентиля по всем пяти эндпойнтам на референсном железе при пустой базе данных и при базе данных, содержащей 100 workflow и 1 000 запусков.

6.2. Испытание автоматической документации REST API

Целью данной методики является подтверждение корректности работы библиотеки `springdoc-openapi` и интерфейса Swagger UI.

В рамках методики в браузере открывается адрес `http://localhost:8080/v1/swagger-ui.html`. Ожидается загрузка интерфейса Swagger UI с заголовком «FluxPilot API» и группировкой REST-эндпойнтов по тегам (Workflows, Versions, Snapshots, Runs, Triggers, Webhooks, Analytics). Раскрывается описание эндпойнта `POST /v1/workflows/{id}/runs`; проверяется наличие текстового описания, схем тела запроса и ответа, примеров. Нажимается кнопка «Try it out», вводится валидный идентификатор workflow и выполняется команда «Execute». Ожидается получение корректного ответа от серверной части. Дополнительно открывается адрес `http://localhost:8080/v1/v3/api-docs`; ожидается возврат валидной JSON-спецификации OpenAPI 3.0. Спецификация импортируется в инструмент Postman, Insomnia или `openapi-generator`; ожидается успешный импорт без ошибок.

Изм.	Лист	№ докум.	Подп.	Дата
RU.17701729.09.03-04 51 03-1				
Инв. № подл.	Подп. и дата	Взам. Инв. №	Инв. № дубл.	Подп. и дата

Критерием успешного прохождения методики является корректная работа интерфейса Swagger UI, документирование всех REST-эндпойнтов, валидность спецификации.

6.3. Испытание целевого покрытия по JaCoCo

Целью данной методики является подтверждение достижения целевого покрытия не менее 90% по строкам и не менее 95% по критическим пакетам.

В рамках методики выполняется команда `cd backend && mvn verify`. По завершении открывается файл `backend/target/site/jacoco/index.html` в браузере. В строке Total таблицы Element фиксируются значения столбцов Instructions Cov. и Branches Cov. Дополнительно фиксируются процентные показатели покрытия для критических пакетов: `run`, `executor`, `blob`, `workflow`, `triggers`.

Критерием успешного прохождения методики является общее покрытие по строкам не ниже 90% (фактическое значение составляет не менее 90%) и покрытие критических пакетов не ниже 95%.

Изм.	Лист	№ докум.	Подп.	Дата
RU.17701729.09.03-04 51 03-1				
Инв. № подл.	Подп. и дата	Взам. Инв. №	Инв. № дубл.	Подп. и дата

7. СВОДНЫЕ ПОКАЗАТЕЛИ

Показатель	Значение
Модульные тесты серверной части (JUnit Jupiter)	около 55
Интеграционные тесты (Testcontainers)	около 12
Доля успешно проходящих тестов	100 %
Покрывание кода по JaCoCo (строки)	не менее 90 %
Покрывание критических пакетов (run/executor/blob)	не менее 95 %
95-й перцентиль REST API на чтение	не более 500 мс
Время от приёма триггера до первого узла running	не более 5 с
Время полного прогона CI-пайплайна backend	не более 8 минут
Готовность системы после docker compose up -d	не более 60 с

Таблица 2. Сводные целевые показатели индивидуальной части

Изм.	Лист	№ докум.	Подп.	Дата
RU.17701729.09.03-04 51 03-1				
Инв. № подл.	Подп. и дата	Взам. Инв. №	Инв. № дубл.	Подп. и дата

8. ПРИЛОЖЕНИЕ. ТРАССИРОВКА ИНДИВИДУАЛЬНЫХ ТРЕБОВАНИЙ К РАЗДЕЛАМ ПМИ

Требование ТЗ	Раздел ПМИ	Метод проверки
Триггеры (ТЗ-2.1.1)	5.1	Ручной и автоматический интеграционный
HTTP-узел (ТЗ-2.1.2)	5.2	Ручной и интеграционный
Узлы потоков данных (ТЗ-2.1.3)	5.3	Модульные тесты класса DataflowNodeExecutorsTest
Code Node и песочница (ТЗ-2.1.4)	5.4	Ручной (функциональные и атакующие сценарии)
Параллельное исполнение (ТЗ-2.1.5)	5.5	Ручной и интеграционный
Версионирование и снапшоты (ТЗ-2.1.6, ТЗ-2.1.13)	5.6	Ручной с прямыми SQL-проверками и интеграционный
WebSocket-трансляция (ТЗ-2.1.10, серверная)	5.7	Ручной через STOMP-клиент
Контент-адресуемое хранение в MinIO (ТЗ-2.1.11)	5.8	Ручной с прямыми SQL-проверками и интеграционный
А/В-аналитика (ТЗ-2.1.12)	5.9	Модульные и интеграционные тесты
Производительность	6.1	Серия из 50 замеров на каждый энд-пойнт
Автоматическая документация API	6.2	Проверка интерфейса Swagger UI
Покрывание тестами	6.3	Отчёт JaCoCo

Таблица 3. Трассировка

Изм.	Лист	№ докум.	Подп.	Дата
RU.17701729.09.03-04 51 03-1				
Инв. № подл.	Подп. и дата	Взам. Инв. №	Инв. № дубл.	Подп. и дата

СПИСОК ИСПОЛЬЗОВАННЫХ ИСТОЧНИКОВ

1. JUnit 5 User Guide [Электронный ресурс]. — URL: <https://junit.org/junit5/docs/current/user-guide/> (дата обращения: 08.04.2026).
2. Testcontainers for Java [Электронный ресурс]. — URL: <https://java.testcontainers.org/> (дата обращения: 12.04.2026).
3. JaCoCo Code Coverage Library [Электронный ресурс]. — URL: <https://www.jacoco.org/jacoco/trunk/doc/> (дата обращения: 15.04.2026).
4. Spring Boot Test Reference [Электронный ресурс]. — URL: <https://docs.spring.io/spring-boot/docs/current/reference/htmlsingle/#features.testing> (дата обращения: 05.04.2026).
5. springdoc-openapi Documentation [Электронный ресурс]. — URL: <https://springdoc.org/> (дата обращения: 05.02.2026).
6. Docker Engine Security Documentation [Электронный ресурс]. — URL: <https://docs.docker.com/engine/security/> (дата обращения: 22.03.2026).

Изм.	Лист	№ докум.	Подп.	Дата
RU.17701729.09.03-04 51 03-1				
Инв. № подл.	Подп. и дата	Взам. Инв. №	Инв. № дубл.	Подп. и дата

ЛИСТ РЕГИСТРАЦИИ ИЗМЕНЕНИЙ

[illegible]